Tectonic: Bridging Synthetic and Real-World Workloads for Key-Value Benchmarking

Alexander H. Ott Shubham Kaushik Boao Chen Subhadeep Sarkar

Problem

Key-Value (KV) Stores

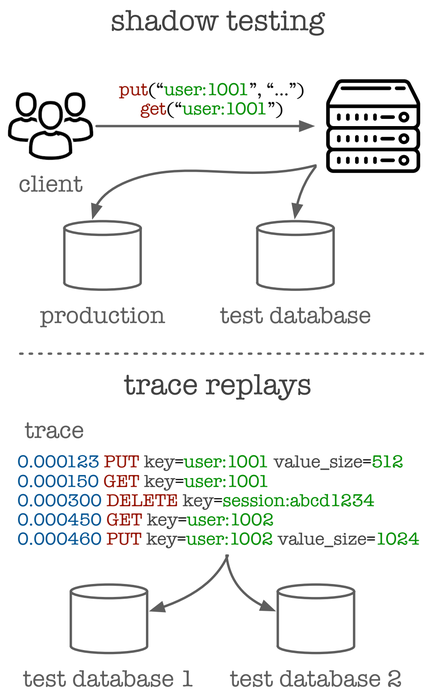
KV-stores are Everywhere

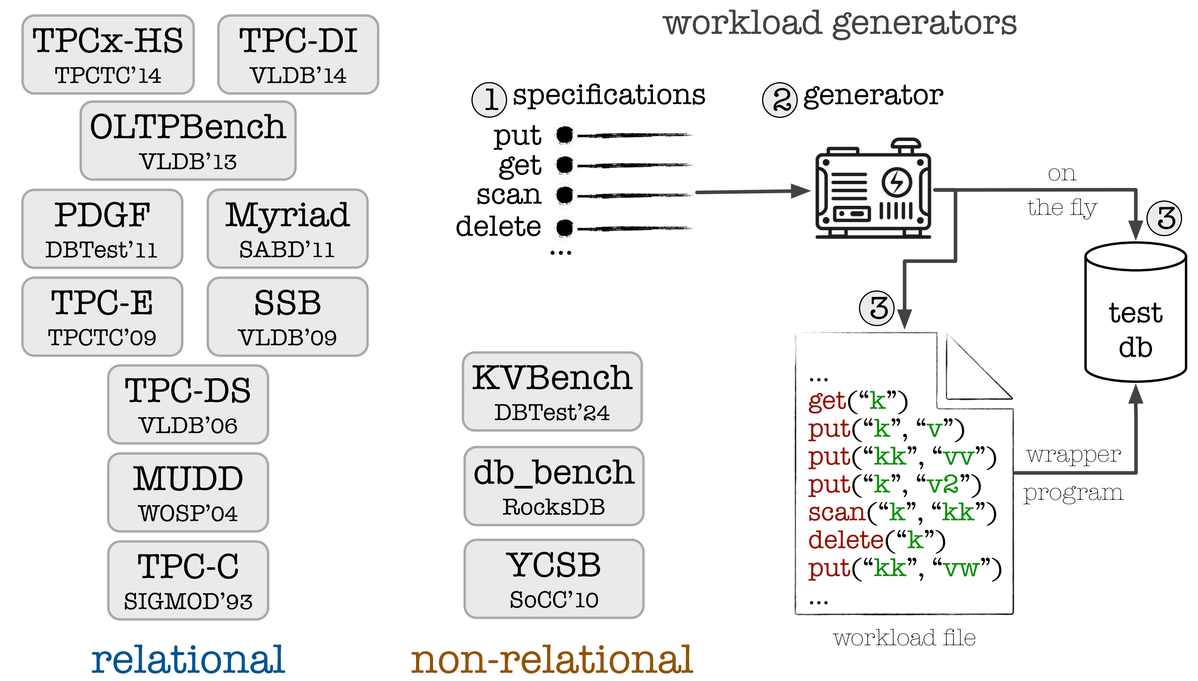
RocksDB, cassandra, levelDB, APACHE HBASE, SQLite, speedb, BigTable, CockroachDB, WT, SCYLLA, influxdb, FoundationDB

Many database vendors!

How to **compare** & **choose** the **best** ?

Benchmark Databases





TPCx-HS TPCTC'14 · TPC-DI VLDB'14 · OLTPBench VLDB'13 · PDGF DBTest'11 · Myriad SABD'11 · TPC-E TPCTC'09 · SSB VLDB'09 · TPC-DS VLDB'06 · MUDD WOSP'04 · TPC-C SIGMOD'93

relational

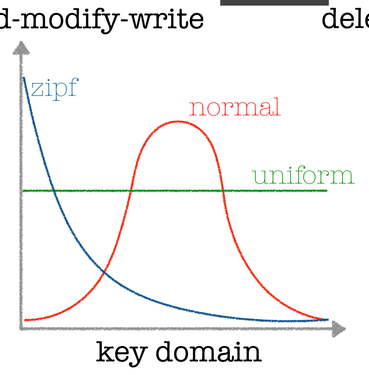
KVBench DBTest'24 · db_bench RocksDB · YCSB SoCC'10

non-relational

Problem

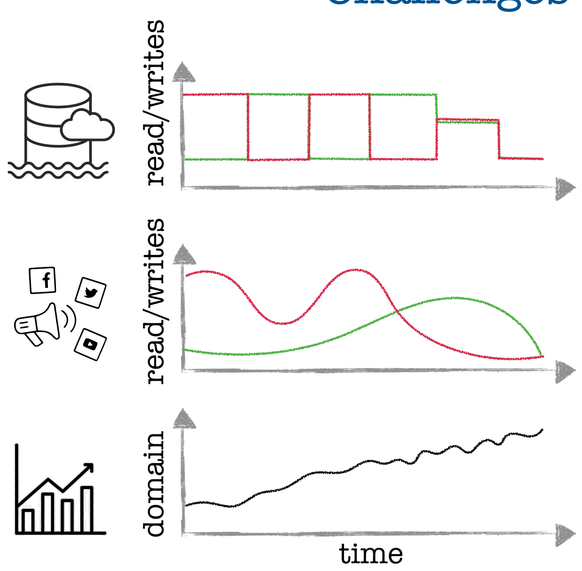
KV Workload Generators

insert · update · point query · range query · read-modify-write · delete · range delete · empty point query · empty point delete



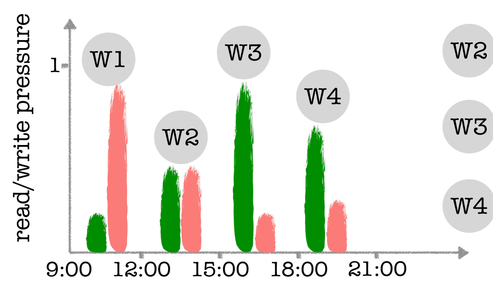
offers ops specific distribution

Challenges to Mimic Real World Workloads



different workload scenarios per application

1 shifting workloads



W1 write-heavy · W2 balanced · W3 read-heavy · W4 mostly-read

human in loop

W_i is agnostic to W_j

not a real workload

2 nearly sorted workloads

t1: e(...)
t3: e(...)
t2: e(...)
.
t99: e(...)

data warehouse (TPC-H)

YCSB/KVBench cannot generate nearly-sorted data

3 variable key-value length

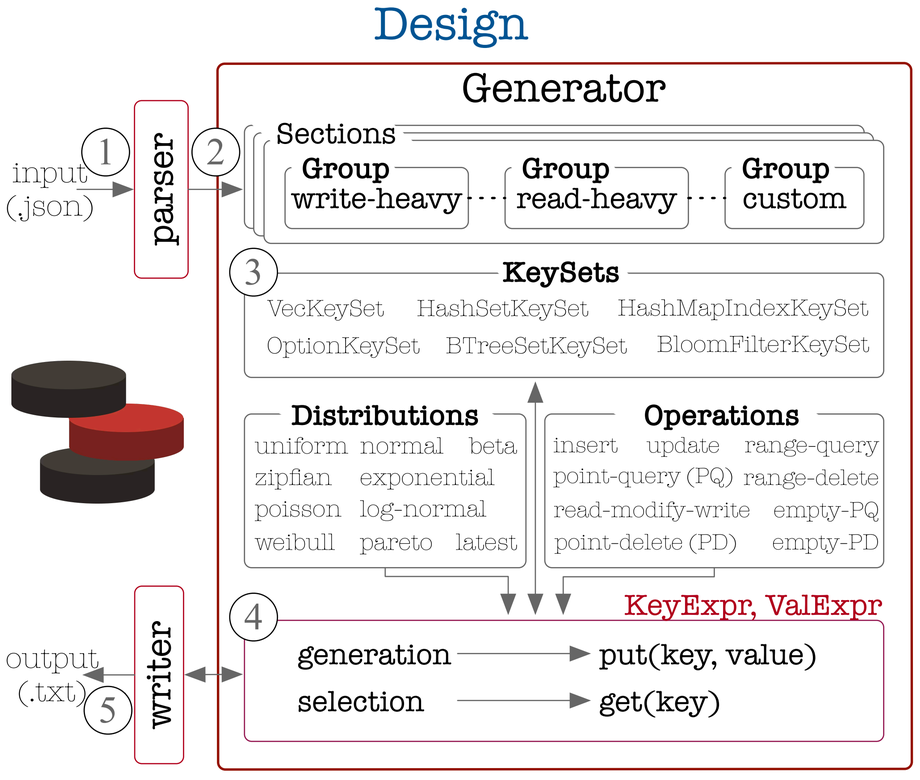
<user_id>:<model_version>:<feature_group>:<timestamp>

4 popular prefix workloads

limited key-value structural control in YCSB/KVBench

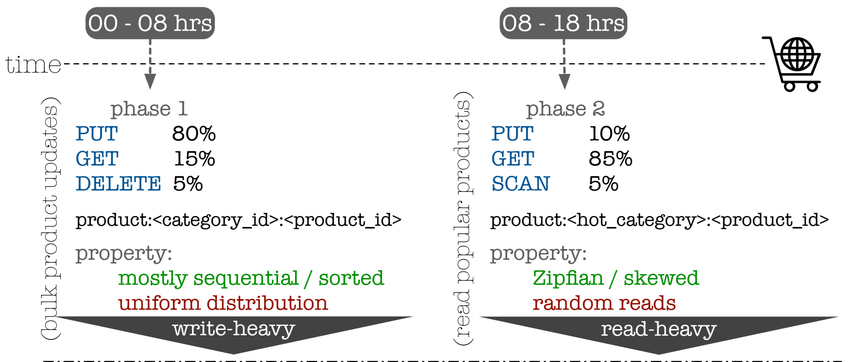
Tectonic

Design



Features

1 shifting workloads



Section

Group 1

```
{"put": {
   "op_count": 1000000,
   "key": {
      "segmented": {
      "separator": ":",
      "segments": [
         "product",
         {"uniform": {"len": 4}},
         {"uniform": {"len": 8}}
      ]}},
   "val": {"uniform": {"len": 128}}},
   "sorted": {"k": 100, "l": 10}
}, "get": {...}, "delete": {...}}
```

Group 2

```
{"get": {
   "op_count": 800000,
   "selection": {"zipf": {
      "n": 1000000, "s": 1.1}}},
"scan": {
   "op_count": 50000,
   "selection": {"zipf": {
      "n": 1000000, "s": 1.1}},
   "selectivity": 0.001},
"put": {
   "op_count": 100000,
   "selection": {"zipf": {
      "n": 1000000, "s": 1.1}},
   "val": {"uniform": {"len": 128}}}}
```

2 offers different **KeySets (KS)**

VecKS
- ✓ blind inserts
- ✓ update
- ✓ point queries
- ✓ range queries
- ✗ empty queries
- ✗ deletes

BloomFilterKS
- ✓ empty deletes
- ✓ empty queries
- ✓ empty queries
- ✗ non-empty deletes

HashMapIndexKS
- ✓ non-empty deletes
- ✗ range queries
- ✗ updates

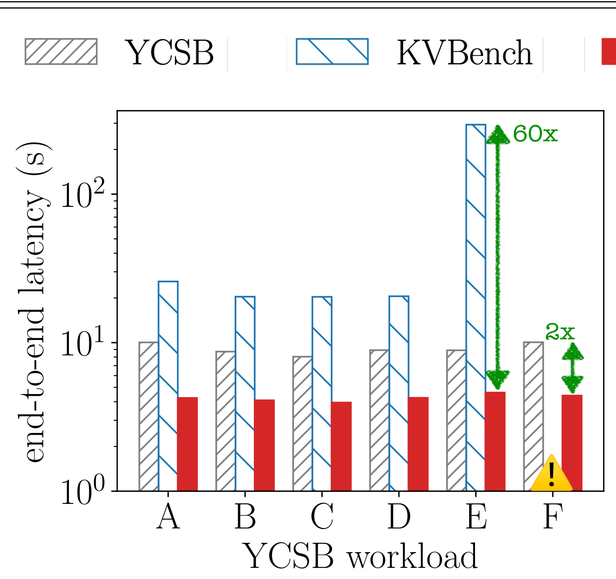
OptionKS
- ✓ non-empty deletes
- ✓ range deletes
- ✓ updates

3 **granular control** on key-value

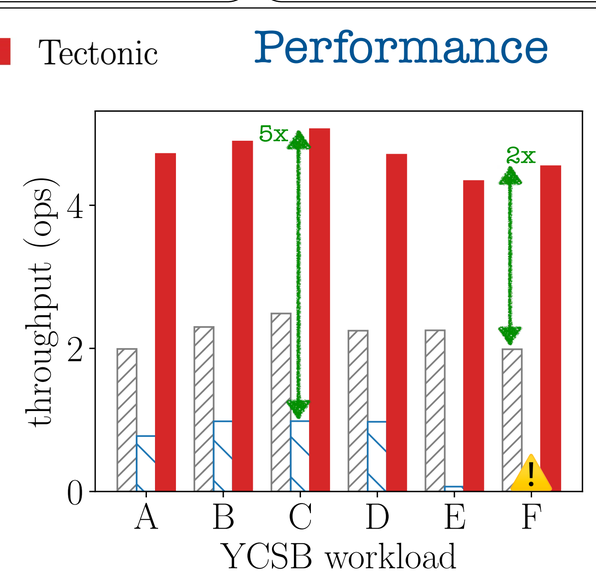
4 varied **sortedness**

Evaluation

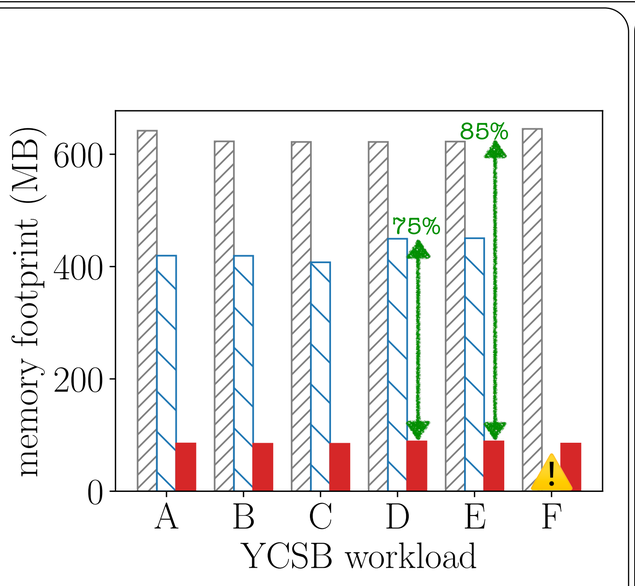
Performance



Tectonic is up to **60x faster** than KVBench and **2x faster** than YCSB

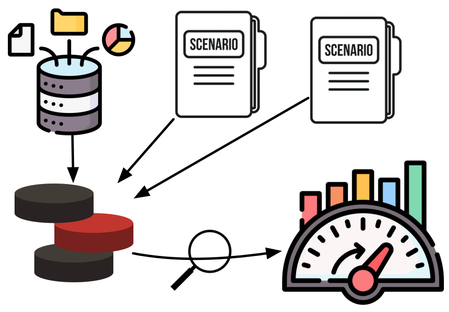


Tectonic is **5x more efficient** than KVBench and **2x** than YCSB



Tectonic uses 75% to 85% less memory than state-of-the-art

Future Work



Compare databases with modern workloads

Integrating Tectonic with Mongodb, Cassandra, and Redis

Brandeis UNIVERSITY
Michtom School of Computer Science